<table>
<tr><td rowspan="1"></td><td colspan="2">Marwadi University
Faculty of Technology
Department of Information and Communication Technology</td></tr>
<tr><td>Subject: Artificial Intelligence (01CT0616)</td><td colspan="2">Aim: To remember the important tasks and forget the less important tasks using LSTM</td></tr>
<tr><td>Experiment No: 05</td><td>Date:25-04-2026</td><td>Enrolment No:92301733024</td></tr>
</table>

Aim: To remember the important tasks and forget the less important tasks using LSTM

IDE: Google Colab

Theory:

Countless learning tasks require dealing with sequential data. Image captioning, speech synthesis, and music generation all require that models produce outputs consisting of sequences. In other domains, such as time series prediction, video analysis, and musical information retrieval, a model must learn from inputs that are sequences. These demands often arise simultaneously: tasks such as translating passages of text from one natural language to another, engaging in dialogue, or controlling a robot, demand that models both ingest and output sequentially-structured data.

Recurrent neural networks (RNNs) are deep learning models that capture the dynamics of sequences via *recurrent* connections, which can be thought of as cycles in the network of nodes. This might seem counterintuitive at first. After all, it is the feedforward nature of neural networks that makes the order of computation unambiguous. However, recurrent edges are defined in a precise way that ensures that no such ambiguity can arise. Recurrent neural networks are *unrolled* across time steps (or sequence steps), with the *same* underlying parameters applied at each step. While the standard connections are applied *synchronously* to propagate each layer's activations to the subsequent layer *at the same time step*, the recurrent connections are *dynamic*, passing information across adjacent time steps. As the unfolded view reveals, RNNs can be thought of as feedforward neural networks where each layer's parameters (both conventional and recurrent) are shared across time steps. Like neural networks more broadly, RNNs have a long discipline-spanning history, originating as models of the brain popularized by cognitive scientists and subsequently adopted as practical modeling tools employed by the machine learning community.

The name of LSTM refers to the analogy that a standard RNN has both "long-term memory" and "short-term memory". The connection weights and biases in the network change once per episode of training, analogous to how physiological changes in synaptic strengths store long-term memories; the activation patterns in the network change once per time-step, analogous to how the moment-to-moment change in electric firing patterns in the brain store short-term memories. The LSTM architecture aims to provide a short-term memory for RNN that can last thousands of timesteps, thus "long short-term memory".

A common LSTM unit is composed of a cell, an input gate, an output gate and a forget gate. The cell remembers values over arbitrary time intervals and the three gates regulate the flow of information into and out of the cell. LSTM networks are well-suited to classifying, processing and making predictions based on time series data, since there can be lags of unknown duration between important events in a time series. LSTMs were developed to deal with the vanishing gradient problem that can be encountered when training traditional RNNs. Relative insensitivity to gap length is an advantage of LSTM over RNNs, hidden Markov models and other sequence learning methods in numerous applications.

Methodology:

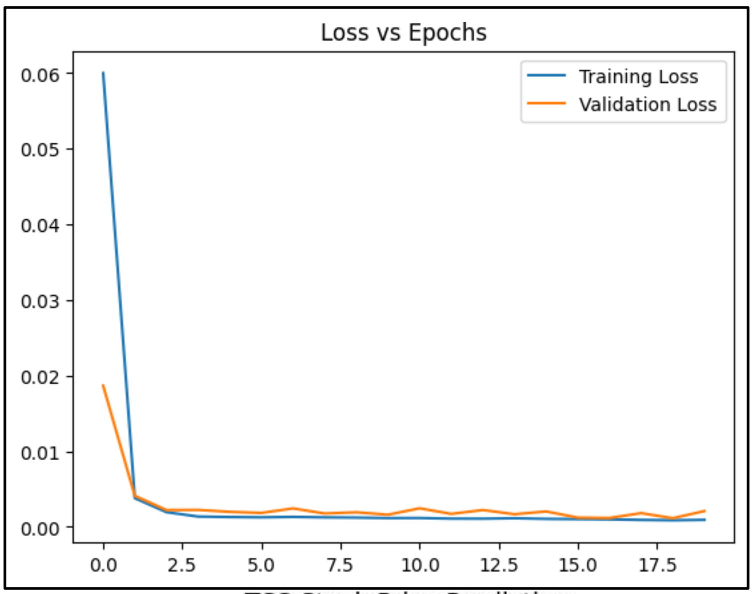
1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Apply LSTM Model
5. Apply the training over the dataset to minimize the loss
6. Observe the cost function vs iterations learning curve

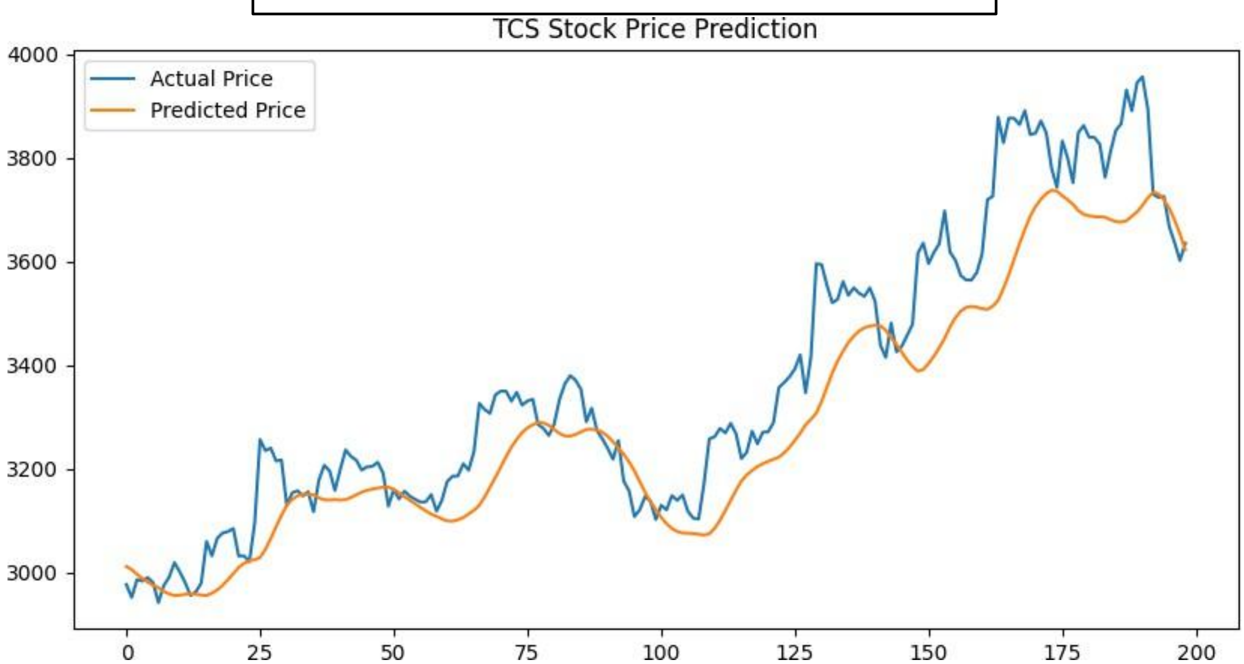
Program (Code):

```
!pip install yfinance

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import yfinance as yf

from sklearn.preprocessing import MinMaxScaler

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, LSTM

df = yf.download("TCS.NS", start="2020-01-01", end="2024-03-31")
print(df.head())

data = df[['Close']].values

scaler = MinMaxScaler(feature_range=(0,1))

scaled_data = scaler.fit_transform(data)

X = []

Y = []
time_step = 60
for i in range(time_step, len(scaled_data)):

   X.append(scaled_data[i-time_step:i,  0])
```

```
    Y.append(scaled_data[i, 0])
X, Y = np.array(X), np.array(Y)

X = X.reshape(X.shape[0], X.shape[1], 1)

split = int(0.8 * len(X))

X_train, X_test = X[:split], X[split:]

Y_train, Y_test = Y[:split], Y[split:]

model = Sequential()

model.add(LSTM(50, return_sequences=True, input_shape=(X.shape[1],1)))

model.add(LSTM(50))

model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')

history = model.fit(X_train, Y_train, epochs=20, batch_size=32, validation_data=(X_test, Y_test)
predicted = model.predict(X_test)

predicted = scaler.inverse_transform(predicted)

actual = scaler.inverse_transform(Y_test.reshape(-1,1))

plt.figure(figsize=(10,5))

plt.plot(actual, label="Actual Price")

plt.plot(predicted, label="Predicted Price")

plt.title("TCS Stock Price Prediction")

plt.legend()

plt.show()

plt.plot(history.history['loss'], label='Training Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Loss vs Epochs")

plt.legend()

plt.show()
```

```
/tmp/ipykernel_8975/3073440315.py:1: FutureWarning: YF.download() has changed argument auto_adjust default to True
  df = yf.download("TCS.NS", start="2020-01-01", end="2024-03-31")
[*********************100%***********************]  1 of 1 completedPrice           Close        High         Low        Open   Volume
Ticker          TCS.NS       TCS.NS       TCS.NS       TCS.NS   TCS.NS
Date
2020-01-01  1866.114136  1880.146842  1854.405641  1866.458417  1354908
2020-01-02  1857.547729  1876.746120  1850.273061  1876.746120  2380752
2020-01-03  1894.566650  1913.808125  1863.014297  1863.014297  4655761
2020-01-06  1894.394653  1916.347920  1883.590161  1898.311847  3023209
2020-01-07  1899.044067  1906.619930  1880.060903  1894.438101  2429317
```





Observation and Result Analysis:

a. Nature of the dataset

- The dataset used is TCS stock price data (time-series dataset), It contains daily closing prices from 2020 to 2024. Data is sequential in nature, where each value depends on previous values. Dataset is normalized between 0 and 1 before training.Suitable for sequence prediction using LSTM

b. During Training Process

The LSTM model learns patterns and trends over time.Previous 60 days data is used to predict the next value.Loss function (MSE) decreases gradually with epochs
Model captures:Upward trends,Downward trends
Validation loss may fluctuate due to market randomness.Training improves as epochs increase

c. After the training Process

Model successfully predicts future stock prices.Predicted values are close to actual values.Model captures overall trend but may miss sudden spikes.Some prediction error exists due to:Market volatility,External factors

d. Observation over the Learning Curve

Training loss decreases continuously.Validation loss decreases initially, then stabilizes.If overfitting occurs:Training loss,Validation loss

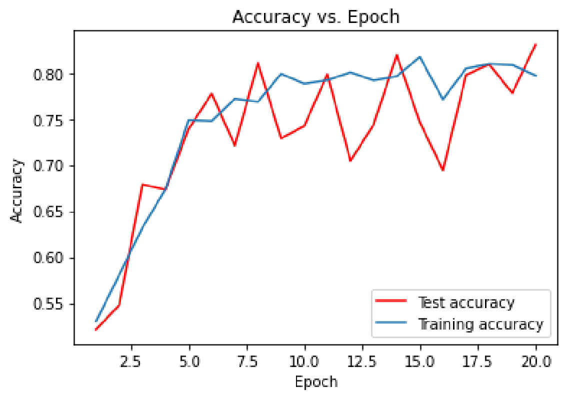
Post Lab Exercise:

a. What is the requirement of LSTM over RNN?

LSTM is required over RNN because:

- RNN suffers from the vanishing gradient problem, making it difficult to learn long-term dependencies
- LSTM uses memory cells and gates (input, forget, output) to retain important information for longer time
- It can handle long sequences effectively
- Provides better performance for time-series and sequential data

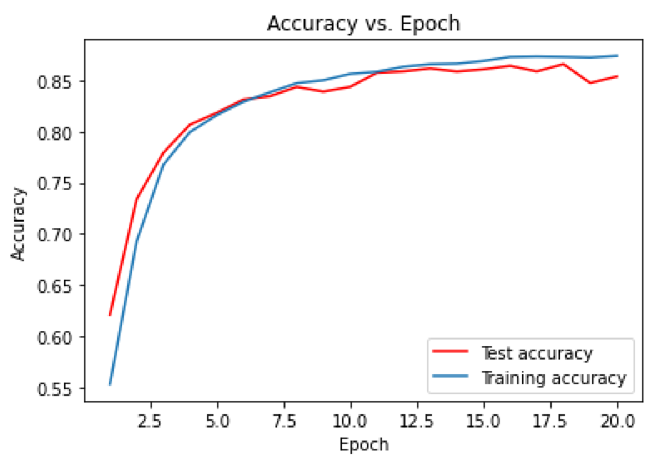
b. Interpretation of graph



What is the meaning of the above graph? How can the graph be smoothened?

- The graph shows training accuracy (blue) and validation/test accuracy (red) vs epochs
- Training accuracy increases steadily
- Validation accuracy is fluctuating a lot

c.



What is the meaning of the above graph? How can the graph be smoothened?

- Both training and validation accuracy are smooth and close
- Accuracy increases steadily and stabilizes
- Very small gap between curves

Post Lab Activity

Obtain the prediction value of the TCS stock for the historical data obtained from the link mentioned as: https://in.investing.com/equities/tata-consultancy-services-historical-data

Select the Date Range from (01-01-2020 to 31-03-2024). Select the price range to be as "DAILY" stock values. Train your LSTM model to obtain the predicted value as close as the actual value. Paste the screenshot of the output.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:25-04-2026**	**Enrolment No:92301733024**